Neural Network Day 1

Python · NumPy 와 AI 수학

HYUNDAI AI Insight Campus · 온디바이스 AI · Neural Network 1/5

2026. 10. 20 (화) 09:00–18:00 · 8교시 · Google Colab

Python | NumPy | 선형대수 | 미분 · 기울기 | 확률 | 경사하강

강사 : 김정인 · 김구수

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1 Colab · AI 개념

" 학습 = 손실을 최소로 만드는 최적화 " 를 설명하고 Colab 노트북을 다룬다

2 Python 속성

C 와 다른 점 — 동적 타입 · 참조 · 리스트 · 딕셔너리 · 클래스를 쓴다

3 NumPy ndarray

shape · axis · keepdims · 브로드캐스팅으로 배열 연산의 결과를 예측한다

4 벡터화 · 선형대수

for 루프 대신 @ 를 쓰고 , X @ W + b 의 shape 를 추적한다

5 미분 · 기울기

중앙 차분과 numerical_gradient 를 구현하고 기울기 방향을 설명한다

6 경사하강 프로젝트

학습률에 따른 수렴 · 진동 · 발산을 실험으로 확인한다

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 오전 도구 (Python · NumPy), 오후 수학 (선형대수 · 미분 · 확률) → 경사하강 프로젝트

교시	시간	주제	형태	애니메이션
1	09:00–09:50	과정 안내 · AI 개념 · Colab 사용법	강의 + 실습	07 Colab 셀 실행 순서
2	10:00–10:50	Python 속성 (C 개발자용)	강의 + 실습	02 변수 참조 모델
3	11:00–11:50	NumPy ndarray — shape · axis · 브로드캐스팅	강의 + 실습	D1-1 axis · D4-1 브로드캐스팅
4	13:00–13:50	벡터화 — for 루프 vs @ · 집계 함수	강의 + 실습	D1-2 행렬곱 추적기
5	14:00–14:50	선형대수 — 내적 · 노름 · 선형 변환	강의 + 실습	D1-3 선형 변환
6	15:00–15:50	미분과 기울기 — 중앙 차분 · numerical_gradient	강의 + 실습	D1-4 수치 미분 · D1-5
7	16:00–16:50	확률 기초 — 분포 · 표준화 · log/exp	강의 + 실습	D1-6 분포 · 표준화
8	17:00–17:50	미니 프로젝트 — 경사하강법 · 정리	개인 실습	D1-5 경사하강 등고선

오늘 사용하는 학습 도구 3 종

예측 → 애니메이션에서 확인 → Colab 노트북에서 코드로 검증 → 워크시트로 자기 점검

강의 노트북

`NN_Day1_lecture.ipynb`

- 교시별 섹션 (## N 교시)
- 개념 → 예제 → 빈칸 ____ → assert 확인
- 빈칸 35 개 · 확인 셀이 ✅ 출력
- 정답본은 수업 후 공개

애니메이션

`anim/index.html`

- 신규 6 개 (D1-1 ~ D1-6)
- 재사용 3 개 (python 과정)
- 프리셋 · 슬라이더로 직접 조작
- 교시 시작 전 예측 → 확인

개념 워크시트

`anim/worksheet.html`

- 교시당 10 문항 × 8 교시 = 80 문항
- 채점 후 해설 · 결과 코드
- 제출 → 오답 정리 캡처
- 노션에 붙여 넣어 제출

예측 → 애니메이션 확인 → 코드 검증 → 워크시트 채점

온라인 자료 링크

브라우저에서 바로 열립니다 — 설치 없음 · 노트북은 "Drive 에 사본 저장" 후 작업

기본 주소 (Day 1)

https://imguru-mooc.github.io/AI_CAMPUS/NN-day1/anim/

index.html	Day 1 애니메이션 목차 (신규 6 + 재사용 3)
worksheet.html	Day 1 개념 워크시트 (80 문항)
NN_Day1_lecture.ipynb	강의 노트북 — Open in Colab
NN_Day1_project.ipynb	8 교시 미니 프로젝트 — Open in Colab

1 과정 안내 · AI 개념 · Colab 사용법

09:00–09:50

학습 목표

- AI ⊃ 머신러닝 ⊃ 딥러닝 관계를 설명한다
- 학습 = 손실을 최소로 만드는 파라미터 찾기
- 5 일 로드맵 : 도구 → 순전파 → 역전파 → 학습 기법 → CNN
- Colab 셀 · 런타임 · 단축키 · ! / % 명령을 쓴다

노트북 실습

1. Open in Colab → Drive 에 사본 저장
2. 환경 셀 실행 · Drive 마운트
3. 연습 1-1 %timeit 측정

▶ 애니메이션 07 Colab 셀 실행 순서 (재사용)

핵심 개념

학습의 정의

```
θ* = argmin L(θ)
# 손실 L 을 가장 작게 만드는 파라미터 θ
```

셀 명령

```
!python --version
!nproc
```

시간 측정

```
%timeit sum(range(1000))
```

환경 셀

```
import numpy as np
rng = np.random.default_rng(42)
drive.mount('/content/drive')
```

체크포인트

런타임이 끊기면 변수가 모두 사라진다 → 환경 셀부터 다시 실행 . 셀 실행 번호가 실제 실행 순서다 .

2 Python 속성 — C 개발자를 위한 정리

10:00–10:50

학습 목표

- 동적 타입 : 변수는 객체를 가리키는 이름
- b = a 는 복사가 아닌 참조 → .copy()
- 리스트 · 딕셔너리 · 함수 · 클래스 (self = this)
- C 코드를 Python 한 줄로 바꿔 쓴다

노트북 실습

1. 문제 2-1 ~ 2-3 합계 · 교환 · 클래스
2. 문제 2-4 참조 vs copy()
3. 문제 2-5 · 보너스 get · 컴프리헨션

▶ 애니메이션 02 변수 참조 모델 (재사용)

핵심 개념

동적 타입

```
x = 3; x = "three"   # 타입은 객체에
2 ** 100              # overflow 없음
```

참조

```
b = a; b.append(4)   # a 도 바뀜
c = a.copy()         # 독립 사본
```

교환 · 합계

```
a, b = b, a
total = sum(range(1, 101))  # 5050
```

딕셔너리

```
counts[w] = counts.get(w, 0) + 1
```

체크포인트

참조 규칙은 NumPy 배열도 같다 — Day 3 에서 W.copy() 를 빼먹으면 원본 가중치가 바뀐다 .

3 NumPy ndarray — shape · axis · 브로드캐스팅

11:00–11:50

학습 목표

- ndim · shape · size · dtype 을 읽는다
- 인덱싱 · 슬라이싱 · reshape(-1) · 불리언 마스크
- axis = " 사라지는 축 ", keepdims 로 크기 1 유지
- 브로드캐스팅 : 뒤 축부터 같거나 1 이면 결합

노트북 실습

1. 인덱싱 · 슬라이싱 출력 예측
2. shape 맞히기 10 문제 (자동 채점)
3. A + b 로 행마다 편향 더하기

▶ 애니메이션 D1-1 axis · keepdims / D4-1 브로드캐스팅

핵심 개념

axis

```
A = np.zeros((3, 4))
A.sum(axis=0).shape   # (4,)
A.sum(axis=1).shape   # (3,)
```

keepdims

```
A.sum(axis=1, keepdims=True).shape  # (3, 1)
```

브로드캐스팅

```
(3,4) + (4,)   → (3,4)   # + 편향
(3,1) + (1,4)  → (3,4)
(3,4) + (3,)   → ValueError
```

마스크

```
A[A > 6]   # Day 3 ReLU mask
```

체크포인트

배열을 받으면 먼저 shape 를 출력한다 — NumPy 오류의 대부분은 모양 문제다 .

4 벡터화 — for 루프 vs @

13:00–13:50

학습 목표

- 반복문이 NumPy 내부 C · BLAS 로 들어간다
- 3 중 for 루프 행렬곱으로 C_ij = Σ A_ik B_kj 이해
- 200×200 행렬곱 속도 비교
- mean · sum · argmax 의 axis 를 고른다

노트북 실습

1. 문제 4-1 matmul_loop 완성
2. for 루프 vs @ · %timeit 측정
3. 문제 4-2 점수표 집계

▶ 애니메이션 D1-2 행렬곱 추적기

핵심 개념

행렬곱 정의

```
C[i][j] += A[i][k] * B[k][j]
```

속도 비교

```
for 루프 : ~580 ms
A @ B    : ~4 ms   → 약 150 배
```

집계

```
scores.mean(axis=0)     # 과목별
scores.sum(axis=1)      # 학생별
np.argmax(total)        # 1 등
```

체크포인트

측정값은 Colab CPU 에 따라 다르지만 수백 배 차이는 같다 . 신경망은 이런 곱셈이 수백만 번이다 .

5 선형대수 — 내적 · 노름 · 선형 변환

14:00–14:50

학습 목표

- 내적 = 두 벡터가 얼마나 같은 방향인가
- 노름 = 벡터 길이 , 코사인 유사도 계산
- 행렬곱 = 공간의 회전 · 확대 · 찌그러짐
- (N,D) @ (D,H) + (H,) = (N,H) shape 규칙

노트북 실습

1. 문제 5-1 코사인 유사도
2. 문제 5-2 회전 · 확대 그리기
3. 문제 5-3 X @ W + b 층 만들기

▶ 애니메이션 D1-3 선형 변환 2×2

핵심 개념

내적 · 노름

```
a @ b              # Σ a_i b_i
np.linalg.norm(a)
```

회전 행렬

```
R = [[cos θ, -sin θ],
     [sin θ,  cos θ]]
Q = P @ R.T
```

신경망 한 층

```
X (4,3) @ W (3,2) + b (2,)  →  Y (4,2)
```

체크포인트

신경망의 한 층은 데이터를 새 공간으로 옮기는 변환이다 — Day 2 XOR 에서 은닉층이 하는 일 .

6 미분과 기울기 — 경사하강의 재료

15:00–15:50

학습 목표

- 미분 = 순간 변화율 , 중앙 차분으로 근사
- h 가 너무 작으면 반올림 오차가 커진다
- 편미분을 모은 벡터 = 기울기 (gradient)
- 기울기는 가장 빨리 커지는 방향 → -∇f 로 내려간다

노트북 실습

1. 문제 6-1 numerical_diff · 접선
2. h 크기별 오차 표
3. 문제 6-2 numerical_gradient · 벡터장

▶ 애니메이션 D1-4 수치 미분 · D1-5 벡터장

핵심 개념

중앙 차분

```
(f(x + h) - f(x - h)) / (2 * h)   # h = 1e-4
```

기울기

```
f = x0² + x1²  →  ∇f = (2x0, 2x1)
(3, 4) 에서 (6, 8)
```

numerical_gradient

```
x[idx] = orig + h; fxh1 = f(x)
x[idx] = orig - h; fxh2 = f(x)
grad[idx] = (fxh1 - fxh2) / (2*h)
x[idx] = orig     # 되돌리기 !
```

체크포인트

numerical_gradient 는 Day 3 역전파 검증 (gradient check) 에 그대로 다시 쓴다 .

7 확률 기초 — 분포 · 표준화 · log/exp

16:00–16:50

학습 목표

- 균등분포 · 정규분포 난수와 히스토그램
- 평균 · 분산 · 표준편차 , 시행 수와 추정 정확도
- 표준화 z = (x - μ) / σ
- log(0) = -inf, exp 큰 값 overflow

노트북 실습

1. 분포 히스토그램 그리기
2. 문제 7-1 · 7-2 주사위 · 표준화
3. -log p 그래프 해석

▶ 애니메이션 D1-6 분포 · 표준화

핵심 개념

난수

```
rng.uniform(-1, 1, n)
rng.normal(0, 1, n)
rng.integers(1, 7, n)   # 주사위
```

확률 추정

```
np.mean(dice == 3)   # ≈ 1/6
```

표준화

```
(x - x.mean()) / x.std()   # 평균 0, 표준편차 1
```

log / exp

```
np.log(0.0)     # -inf  → + eps
np.exp(1000.0)  # inf   → Day 2
```

체크포인트

표준화는 Day 4 BatchNorm, -log p 는 Day 2 Cross-Entropy 로 이어진다 .

8 미니 프로젝트 — 경사하강법으로 최소점 찾기

17:00–17:50

NN_Day1_project.ipynb · $f(x, y) = x^2 + 3y^2$

$p \leftarrow p - \eta \nabla f(p)$ $\quad\quad$ $\nabla f = (2x,\ 6y)$

과제	내용	배점
1	목표 함수 · 해석적 기울기	20
2	수치 기울기와 비교 (오차 < 1e-6)	20
3	gradient_descent → history (steps+1, 2)	30
4	학습률 4 개 경로 비교 그래프	20
5	결과 해석 (강사 채점)	10

채점 셀 실행 → 자동 채점 90 점 + 해석 10 점

▶ D1-5 경사하강 등고선에서 학습률을 먼저 예측해 보고 코드로 확인

학습률별 결과 (출발점 (-4, 2), 30 스텝)

$\eta = 0.01$	느리게 수렴 — 최소점에 못 미침	f = 5.05
$\eta = 0.1$	부드럽게 수렴	f = 2.4e-5
$\eta = 0.3$	y 가 지그재그로 진동하며 수렴	f = 1.8e-5
$\eta = 0.35$	발산 — y 가 매 스텝 1.1 배	f = 3.7e+3

핵심 관찰

$y \leftarrow (1 - 6\eta)\, y \ \rightarrow\ |1 - 6\eta| < 1 \iff \eta < 1/3$
가장 가파른 방향이 학습률 상한을 정한다 → Day 4 Momentum · Adam

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

학습	손실을 최소로 만드는 파라미터 찾기 — 5 일 동안 직접 구현	1 교시
참조	b = a 는 같은 객체 , 독립 사본은 .copy()	02
axis	axis 는 사라지는 축 , keepdims 로 크기 1 유지	D1-1
브로드캐스팅	뒤 축부터 같거나 1 이면 결합 — X @ W + b 의 + b	D4-1
벡터화	for 루프 대신 @ — 수백 배 빠르다	D1-2
선형 변환	행렬곱은 공간을 옮긴다 , (N,D)@(D,H) → (N,H)	D1-3
기울기	중앙 차분 · numerical_gradient, -∇f 방향으로 내려간다	D1-4 · D1-5
표준화	(x - μ) / σ — 입력 정규화와 BatchNorm 의 핵심	D1-6

Day 2 로 가는 다리 — 뉴런 · 순전파 · 손실

오늘 만든 도구가 내일 그대로 신경망이 됩니다

Day 1 (오늘)		Day 2 (내일)
`X @ W + b · 브로드캐스팅`	→	층 = 행렬 순전파 , 3 층 신경망 shape 추적
`np.exp · overflow`	→	Softmax 와 최댓값 빼기 수치 안정화
`np.log · -log p 그래프`	→	Cross-Entropy 손실
`numerical_gradient`	→	로지스틱 회귀를 수치 미분으로 학습
`경사하강 · 학습률`	→	선형 회귀 · 로지스틱 회귀 학습 루프

퍼셉트론 → XOR → 활성화 함수 → 행렬 순전파 → Softmax · Cross-Entropy → 로지스틱 회귀

제출 : 미니 프로젝트 노트북 (채점 셀 결과 포함) · 워크시트 오답 정리 캡처